

Resumen-tema-2-IA.pdf



saracordobaa



Inteligencia Artificial



3º Grado en Ingeniería Informática - Tecnologías Informáticas



Escuela Técnica Superior de Ingeniería Informática
Universidad de Sevilla



[Accede al documento original](#)

CURSO NUEVO EQUIPO NUEVO

CeX
webuy.com



Química con Manuel

Aprende, comprende, avanza



TEMA 2. BÚSQUEDA EN ESPACIO DE ESTADOS PLANIFICACIÓN

1. ESPACIO DE ESTADOS CON EJEMPLO 8-PUZZLE

↳ **ESTADO**: descripción de cualquier posible situación del problema. La representación escogida influye en el número de estados y en la eficiencia de los procedimientos de búsqueda de soluciones.

↳ **ESTADO INICIAL**: situación de partida.

↳ **DESCRIPCIÓN DEL OBJETIVO**: conjunto de estados que llamaremos finales.

UN ÚNICO ESTADO FINAL →

1	2	3
8		4
7	6	5

Descripción de la posición exacta de cada uno de los bloques

2	8	3
7	1	4
6	5	

2	3
1	8
7	6

Representación vs. implementación

- Listas anidadas: $[[2, 8, 3], [7, 1, 4], ["H", 6, 5]]$
- Tuplas: $(2, 8, 3, 7, 1, 4, "H", 6, 5)$
- Diccionarios: $\{\text{primeraizquierda: 2, primeracentro: 8, ...}\}$

Número de estados: $9! = 362.880$

↳ **ACCIONES**: conjunto de opciones que permiten transformar unos estados en otros.

• elementos que describen una acción:

↳ Aplicabilidad: precondición y poscondición

↳ Efecto: transformación que produce una acción sobre un estado.

↳ **SOLUCIÓN DE UN PROBLEMA**: secuencia de acciones a realizar para conseguir el objetivo su aplicación desde el estado inicial obtiene el final.

2	8	3
1	6	4
7	5	

Arriba

2	8	3
1	4	
7	6	5

Arriba

2	3
1	8
7	6

Izquierda

2	3
1	8
7	6

Abajo

1	2	3
8	4	
7	6	5

Derecha

1	2	3
8	4	
7	6	5

8-puzzle:

- Según los movimientos de los bloques: $32 = (8 \text{ bl.} \times 4 \text{ mov.})$
- Según los movimientos del hueco: 4.

Representación

- Arriba
- Derecha
- Abajo
- Izquierda

Implementación

- $(0, -1)$
- $(1, 0)$
- $(0, 1)$
- $(-1, 0)$

Descripción de la primera: Arriba

- Aplicabilidad: es aplicable a estados en los que el hueco no esté en la primera fila
- Efecto: intercambiar las posiciones del hueco y del bloque que está encima de éste

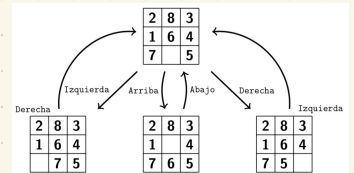
6	8	3
7	2	
1	4	5

→

6	8	
7	2	3
1	4	5

2. ESPACIO DE ESTADOS COMO GRAFO DIRIGIDO

Los vértices son los distintos estados. Los arcos son aquellos que se obtienen a partir de una acción aplicable.



3. ÁRBOLES DE BÚSQUEDA Componentes: estado, padre y acción

Elementos que definen un problema

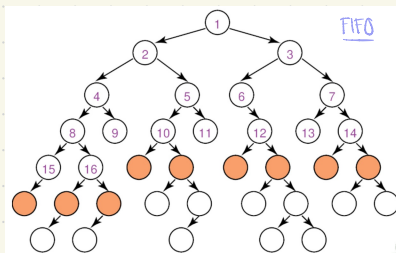
- **estado_inicial**: almacena la representación del estado inicial
- **es_estado_final (ESTADO)**: comprueba si un estado es final
- **acciones_aplicables (ESTADO)**: devuelve las acciones aplicables a un estado
- **aplicar (ESTADO, ACCIÓN)**: obtiene el resultado de utilizar una acción aplicable sobre un estado

búsqueda.genérica()

1. $\text{abiertos} \leftarrow \{\text{estado : estado.inicial}\}$
 $\text{cerrados} \leftarrow \emptyset$
2. Mientras abiertos no esté vacía,
 - 2.1 $\text{actual} \leftarrow \text{primer nodo de abiertos}$
 $\text{abiertos} \leftarrow \text{resto de abiertos}$
 $\text{cerrados} \leftarrow \text{cerrados} \cup \{\text{estado(actual)}\}$
 - 2.2 Si **es_estado_final**(estado(actual)),
 - 2.2.1 Devolver el camino desde la raíz a actual
 - 2.2.2 en caso contrario
 - 2.2.2.1 $\text{nuevos.sucesores} \leftarrow \text{nodos de sucesores(actual)}$
cuyo estado no está en ningún nodo de abiertos, ni en cerrados
 - 2.2.2.2 $\text{abiertos} \leftarrow \text{abiertos} \cup \text{nuevos.sucesores}$
3. Devolver Fallo.

Cerrado: estados de los nodos YA VISITADOS

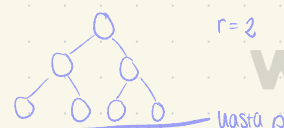
3.1. BÚSQUEDA EN ANCHURA. BFS recorre nivel por nivel. Primero: todos los nodos del estado inicial, luego los nodos de esos nodos,...



- ✓ - garantiza el camino más corto en n° de pasos.
- Es completa: si existe una solución, la encontrará.
- Es adecuada: encuentra la mejor sol. en coste.
- ✗ - alto consumo de memoria. Debe almacenar todos los nodos generados en niveles anteriores.
- lento para problemas grandes. No adecuada para espacios infinitos

- N° de nodos que visitará hasta encontrar la solución:

$$O(r^{p+1}) = \frac{r^{p+1} - 1}{r - 1} \quad \left\{ \begin{array}{l} p = \text{profundidad de la sol. encontrada} \\ r = \text{factor de ramificación} \end{array} \right.$$



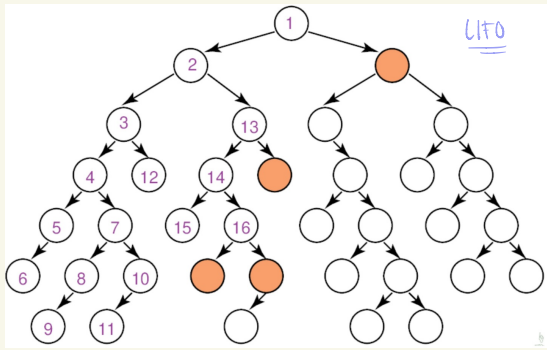
WUOLAH



Escanéame

Made with Goodnotes

3.2. BUSQUEDA EN PROFUNDIDAD DFS: recorre el grafo siguiendo un camino hasta el final, antes de retroceder



Explora un camino completo antes de retroceder

- ✓ - melhor controlo de memória. Só adiciona los nodos del camino actual y los vecinos
 - pode encontrar solucoes ou explorar todo el espacio.
-
- ✗ - No es completa. Si hay ciclos o ramias infinitas pode quedarse atropada.
 - No es adecuada (optima). Pode encontrar una sol. mas larga.

$$O(r^{m+1}) = \frac{r^{m+1} - 1}{r - 1} \quad \left\{ \begin{array}{l} r: \text{factor multiplicación} \\ m: \text{máxima profundidad de la búsqueda (puede ser finita)} \end{array} \right.$$

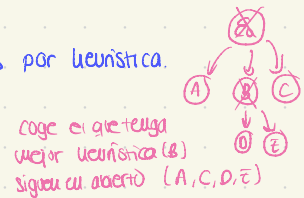
↳ DFS ACOTADA: no explora más allá de una profundidad (cota) máxima.
Es adecuada pero no completa.

- ↳ **DFS ITERATIVA**: ejecuta varias búsquedas DFS acotada, pero aumentando el límite de profundidad cada vez.

3.3. BÚSQUEDA POR PRIMERO EL MEJOR: HEURÍSTICAS.

Asuma preferentemente los nudos con heurística más baja. Ordene la cola de abiertos por heurística. Siempre elige expandir lo que estima que está más "cerca" del objetivo.

- No es completa: una mala heurística podría hacer que se tomara un camino infinito.
- No es minimal: no garantiza soluciones con el menor n° de acciones.



4. PLANIFICACIÓN

Suponemos: - u° finito de estados - el ruido no cambia - ruido cerrado: aquella inf. que no tenemos la suponemos falsa

4.1. FORMALISMO LÓGICO: LINGUAGEM PDDL

- ↳ lenguaje para representar problemas:

- Ctes : objetos del mundo (en MANUSC)
- Ubles : objetos genéricos (HINUSC)
- Simbolos de predicados para expresar propiedades
- Simbolos para representar acciones

- Terminología:
 - átomos: símbolos de predicados que pueden ser verdadero o falso $P(x, y)$
 - literales: átomos

- ↳ **Estados:** conjunciones de predicados positivos. Damos por echo mundo cerrado
- ↳ **la Objetivo:** conjuncion de predicados tanto pos. como negativas

↳ Todo estado representa un objetivo. Un estado que solo tiene predicados positivos puede ser un objetivo. No todo objetivo son estados: si tienen predicados negativos, ya no pueden ser estados.



- Descripción de bu estado:

DESPEJADO (B)	BRAZOLIBRE (C)
DESPEJADO (C)	SABRE (B.A)
DESPEJADO (D)	SABRENTSA (D)
SABRENTSA (C)	SABRENTSA (A)

- Predicados lógicos:

DISPUESTO (X)	SOBRE (x,y)
BRAZOLIBRE ()	ENGARRADO (X)
SOBREMITA (X)	

4.2. REPRESENTACIÓN DE OBJETIVOS descripción de los estados finales

Un estado satisface un objetivo si es posible sustituir los vótes por ctes de manera que el estado contenga todos los literales positivos instanciados y ninguno de los negativos.

Todos los estados que cumplan con el objetivo son finales.

EJEMPLO: $\text{SOBRE}(B, A)$, $\text{SOBREMESA}(A)$, $\neg \text{SOBRE}(C, D) \leftarrow \text{OBJETIVO}$

4.3. DESCRIPCIÓN DE ESQUEMAS DE ACCIONES

se especifica lo que cambia por la acción lo que no se puede cambiar igual. Se describe mediante:

- Colocar un bloque sobre otro:

APILAR(x,y)
 Prec.: DESPEJADO(y), AGARRADO(x)
 Efec.: -DESPEJADO(y), -AGARRADO(x),
 BRAZOLIBRE(), SOBRE(x,y), DESPEJADO(x)

- Quitar un bloque que estaba sobre otro:

DESAPILAR(x,y)
 Prec.: SOBRE(x,y), DESPEJADO(x), BRAZOLIBRE()
 Efec.: -SOBRE(x,y), -DESPEJADO(x), -BRAZOLIBRE(),
 AGARRADO(x), DESPEJADO(y)

- Agarrar un bloque con el robot:

AGARRAR(x)
 Prec.: DESPEJADO(x), SOBRELAMESA(x), BRAZOLIBRE()
 Efec.: -DESPEJADO(x), -SOBRELAMESA(x), -BRAZOLIBRE(),
 AGARRADO(x)

- Bajar un bloque hasta la superficie:

BAJAR(x)
 Prec.: AGARRADO(x)
 Efec.: -AGARRADO(x),
 SOBRELAMESA(x), BRAZOLIBRE(), DESPEJADO(x)

↳ Su nombre y roles involucradas

↳ Precondición: a cumplirse para aplicar acción

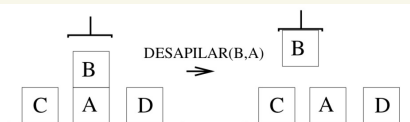
↳ Efectos: cambios que se producirán

Positivos: pasarán a ser ciertos
 Negativos: pasarán a ser falsos

4.4. APLICACIÓN DE ACCIÓN

↳ PLAN: secuencia de acciones.

↳ SOLUCIÓN: plan que a partir del estado inicial obtiene un estado que satisface el objetivo



* Estado antes de aplicar DESAPILAR(B,A):

$E = \{ \text{DESPEJADO(B)}, \text{DESPEJADO(C)}, \text{DESPEJADO(D)}, \text{BRAZOLIBRE()}, \text{SOBRE(B,A)}, \text{SOBRELAMESA(C)}, \text{SOBRELAMESA(D)}, \text{SOBRELAMESA(A)} \}$

* Precondiciones de DESAPILAR(B,A):

Prec = {SOBRE(B,A), DESPEJADO(B), BRAZOLIBRE() }

----- Condiciones satisfechas en el estado -----
 (acción aplicable)

* Efectos de DESAPILAR(B,A):

Efec- = {SOBRE(B,A), DESPEJADO(B), BRAZOLIBRE() } in q quitamos

Efec+ = {AGARRADO(B), DESPEJADO(A)} no añadimos

* Estado después de aplicar DESAPILAR(B,A):

$E' = (E \setminus \text{Efec-}) \cup \text{Efec+} = \{ \text{DESPEJADO(C)}, \text{DESPEJADO(D)}, \text{SOBRELAMESA(C)}, \text{SOBRELAMESA(D)}, \text{SOBRELAMESA(A)}, \text{AGARRADO(B)}, \text{DESPEJADO(A)} \}$

5. BÚSQUEDA EN EL ESPACIO DE ESTADOS

5.1. BÚSQUEDA HACIA ADELANTE EN PROFUNDIDAD CON HEURÍSTICA: se ordenan por heurística los sucesores del estado actual.

- Adecuado, completo, siempre termina. No se garantiza la sol. más corta
- Su eficiencia depende de la heurística. Si es buena, habrá mucha ramificación



no tengo lista de abiertos, es un proceso recursivo.

En cada paso miro cuál es el mejor sucesor y vuelvo a aplicar el mismo proceso recursivamente.

No puedo volver hacia atrás hasta no agotar la rama.

5.2. BÚSQUEDA HACIA ATRÁS: se empieza en el objetivo, se aplican las acciones a la inversa para que acciones me hubieran llevado a él. Solo aplicar hacia atrás las acciones relevantes para el objetivo: una acción es relevante si podría ser la última de un plan que llevara al objetivo.

↳ Acciones relevantes → si al menos uno de los efectos de la acción está en el objetivo con el mismo signo.

↳ si ninguno de los efectos de la acción aparece con distinto signo.

↳ BÚSQUEDA HACIA ATRÁS CON HEURÍSTICA: incorpora heurística para ordenar objetivos.



Química con Manuel

Aprende, comprende, avanza



Llevo desde 1995 acompañando y ayudando a estudiantes como tú. Me apasiona mi labor y sobre todo la química

6 HEURÍSTICA PARA PLANIFICACIÓN BASADAS EN ESPACIOS DE ESTADOS buscamos heurísticas independientes del objetivo.

↳ Heurísticas basadas en relajar el problema:

Algunas ideas para relajar el problema:

- Ignorar las precondiciones y/o efectos negativos de las acciones
- Suponer que cada literal de un objetivo se alcanza de manera independiente: como si el número de acciones necesarias para alcanzar un objetivo fuera la suma del número de pasos necesario para alcanzar cada literal del objetivo
- Suponer que el número de acciones necesarias para alcanzar un objetivo es el número de pasos máximo necesario para alcanzar uno de sus literales
- Ignorar todas las precondiciones de las acciones
- Ignorar determinados predicado

(predicado)

↳ la HEURÍSTICA Δ_0 Dado un estado e , un átomo p y un objetivo g :

→ si p aparece en e , $\Delta_0(e, p) = 0$

→ si p no aparece en e ni en los efectos positivos de ninguna acción, $\Delta_0(e, p) = +\infty$

→ si p no aparece en e pero sí como efecto positivo de alguna acción, $\Delta_0(e, p)$ es el menor costo de todas las acciones que lo generan, contando 1 por la acción más lo que cuesta conseguir sus precondiciones.

→ El coste de lograr todo el objetivo g desde el estado e es la suma de los costes de conseguir cada literal que forma el objetivo.

EJEMPLO →

• Acciones:

A	B	C	D
Pre: P1 Efe: P5	Pre: P1 Efe: -P2, -P5, P3, P6	Pre: P5, P6 Efe: -P8, P7	Pre: P3 Efe: P8

• Estado inicial: {P1, P2}

• Objetivo: {P7, P8}

Aplicando propiamente hacia adelante:

- Estado inicial → {P1, P2}
las acciones aplicables son A y B
1- {P1, P2, P5} 2- {P1, P2, P3}

- Exploramos según heurística:

$$1- \Delta_0(\{P1, P2, P5\}, \{P7, P8\}) = \Delta_0(\{P1, P2, P5\}, \{P7\}) + \Delta_0(\{P1, P2, P5\}, \{P8\}) = 2 + 2 = 4$$

$$\Delta_0(\{P1, P2, P5\}, P_7) = \Delta_0(\{P1, P2, P5\}, \{P5, P6\}) = \Delta_0(\{P1, P2, P5\}, P_6) = 1 + 1 + \Delta_0(\{P1, P2, P5\}, P_7) = 3$$

quiero P_7 , veo si hay alguna acción que contenga P_7 en sus efectos → C. Sumamos 1 + sus pre.

$$\Delta_0(\{P1, P2, P5\}, P_8) = 1 + \Delta_0(\{P1, P2, P5\}, P_3) = 1 + 1 + \Delta_0(\{P1, P2, P5\}, P_1) = 3$$

* Se calculaba la heurística de la rama B(2) y entraba por la rama con menor heurística.



Escanéame

Made with Goodnotes

WUOLAH